

A project report on

Medicine Finder: A Web-Based Medicine Search and Pharmacy Management System

Submitted in partial fulfillment for the award of the degree of

B.Tech

Computer Science and Engineering with Information Security

by

RIDANSHI AGARWAL (23BCI0026)

Under the Guidance of

Dr. KONATHAM SUMALATHA

Assistant Professor Sr. Grade 1



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2025

ABSTRACT

In today's fast-paced and densely populated world, access to essential medicines remains a critical challenge, especially in cases of medical emergencies or in areas with limited pharmacy availability. This project, "**Medicine Finder**," presents the design and implementation of an intelligent, location-aware web-based system aimed at bridging the gap between patients and nearby pharmacies. The system allows users to instantly search for required medicines based on symptoms or specific drug names and provides detailed information about local medical stores, including availability, price, and contact details. Unlike traditional e-pharmacy models that rely heavily on home delivery and central stock systems, Medicine Finder focuses on *proximity-based discovery*, enabling users to obtain medicines quickly without delays caused by delivery logistics. The platform features a Node.js and Express backend integrated with a MongoDB database for efficient storage and retrieval, while the React-based frontend ensures a seamless and responsive user experience. The system also supports advanced features such as medicine comparison, type-based filtering, and an admin-controlled pharmacy registration module. Through its focus on accessibility, accuracy, and immediacy, Medicine Finder successfully transforms the process of medicine procurement into a faster, smarter, and more reliable experience for both urban and rural users.

CONTENTS

Title	Page No
CHAPTER 1: INTRODUCTION	4
1.1 Introduction	4
1.2 Theoretical Background	4
1.3 Motivation	5
1.4 Aim of the proposed Work	6
1.5 Objective(s) of the proposed work	6
1.6 Report Organization	6
CHAPTER 2: LITERATURE SURVEY	7
2.1 Survey of the Existing Models/Work	7
2.2 Gaps Identified in the Survey	8
2.3 Problem Statement	9
CHAPTER 3: OVERVIEW OF THE PROPOSED SYSTEM	9
CHAPTER 4: REQUIREMENTS ANALYSIS AND DESIGN	10
4.1. Requirements Analysis	10
4.1.1. Functional Requirements	10
4.1.2. Product Perspective	11
4.1.3. Product Features	11
4.1.4. User Characteristics	12
4.1.5. Assumption & Dependencies	13
4.1.6. Domain Requirements	13
4.1.7. Non-Functional Requirements	14
4.2. System Modeling	14
4.3. Engineering Standard Requirements	15
4.4. System Requirements	16
4.4.1. Hardware Requirements	16

4.4.2. Software Requirements	16
4.5. System Design	17
4.5.1. System Architecture	17
4.5.2. Detailed Design	18
CHAPTER 5: IMPLEMENTATION AND TESTING	20
5.1. Methodology	20
5.2. Dataset Description	20
5.3. Implementation Setup and Screenshots	21
CHAPTER 6: RESULTS AND DISCUSSION	29
6.1. Research Finding	29
6.2. Result Analysis	30
6.3. Evaluation	30
CHAPTER 7: CONCLUSION AND SCOPE FOR FUTURE WORK	31
7.1. Conclusion	31
7.2. Scope for Future Work	31
REFERENCES	32
ANNEXURE I - SAMPLE CODE	34

CHAPTER 1: INTRODUCTION

1.1 Introduction

Access to essential medicines is a fundamental aspect of healthcare, yet many individuals continue to face significant barriers when trying to obtain them promptly. The challenge is especially pronounced in regions where pharmacies are dispersed, inventory data is unavailable, or delivery systems are unreliable. In large metropolitan cities, patients often struggle to locate nearby stores that have a required medicine in stock, while in small towns and rural areas, the lack of digital infrastructure further compounds the issue. The traditional approach to addressing this problem has been through centralized e-pharmacy platforms, where users order medicines online and wait for deliveries. However, this model is not always suitable for urgent medical needs or for users who prefer in-person purchases.

This project, “**Medicine Finder**,” proposes a modern, data-driven solution that bridges this accessibility gap by connecting users directly with local pharmacies. The system enables users to search for medicines based on their names or symptoms and instantly view nearby stores where the medicine is available. Unlike existing systems that rely purely on keyword-based search or delivery logistics, Medicine Finder integrates location awareness, intelligent filtering, and a real-time comparison mechanism to provide a holistic view of medicine availability. Users can also compare up to three medicines simultaneously, helping them make informed purchasing decisions based on price, type, or manufacturer.

The platform’s architecture is built using modern web technologies such as Node.js, Express, MongoDB, and React.js, ensuring scalability, speed, and ease of access. It further introduces an administrative interface where authorized store owners can upload and manage medicine details, while end users can freely search without the need for authentication. In doing so, **Medicine Finder** transforms the conventional pharmacy lookup process into an intelligent, responsive, and community-driven ecosystem that empowers users to obtain medicines quickly, conveniently, and reliably.

1.2 Theoretical Background

The foundational technology that enables this leap from simple search to intelligent medicine discovery is the **integration of real-time data retrieval with location-based intelligence**.

The **Medicine Finder** system is built upon a robust full-stack architecture that unifies the strengths of a web-based frontend, an optimized backend, and a dynamically updating database. It works by compelling the backend server to reference an authoritative, external data source — in this case, the live inventory records of registered medical stores — before generating any output for the user. In the context of this project, the collection of pharmacy databases and medicine inventories serves as that authoritative knowledge base.

This architectural choice directly addresses one of the most significant limitations of traditional e-pharmacy systems: the lack of immediate and verified stock information. By grounding every search query in real-time store data, the system ensures that the results are not only relevant and context-aware but also accurate and reliable. The entire pipeline implements this intelligent retrieval paradigm: efficient querying of medicine availability combined with structured data generation to produce precise, trustworthy, and verifiable responses for end users.

1.3 Motivation

The primary motivation for this project stems from addressing key "User Pain Points" faced by individuals in need of urgent medical assistance. People often struggle to locate nearby pharmacies that have a specific medicine available, especially during emergencies or at odd hours. Medicines are frequently out of stock, and users are left uncertain about where to find alternatives, leading to delays in treatment and unnecessary stress.

Furthermore, traditional e-pharmacy systems and online medicine delivery platforms lack immediacy and transparency. They require users to place an order and wait for delivery, which is not always practical when medicine is needed right away. There is no intelligent system that can instantly locate nearby stores with the required stock, provide contact details, or offer comparative insights based on price and brand.

The goal is to create a unified, intelligent platform that can understand user needs, retrieve accurate pharmacy data, and present results interactively. The project is motivated by the need to transform the conventional, time-consuming medicine search process into an efficient, real-time assistance system, moving beyond static listings to a solution that truly empowers users through accessibility, reliability, and speed.

1.4 Aim of the proposed Work

The aim of this proposed work is to design, build, and deploy a robust **“Medicine Locator and Information System,”** implemented as the **“Medicine Finder – Intelligent Medicine Search and Comparison Platform.”**

This system aims to function as a real-time digital bridge between users and nearby pharmacies, allowing individuals to instantly locate stores that have the required medicines in stock. The primary goal is to provide a tool that can deliver fast, reliable, and comprehensive information about medicine availability, pricing, and alternatives, enabling users to make informed decisions without delays. By combining location awareness, real-time database connectivity, and intelligent filtering, the system ensures that every search result is accurate, relevant, and immediately actionable.

1.5 Objective(s) of the proposed work

The main objective of this project is to design and develop an intelligent Medicine Finder System that simplifies the process of finding and managing medicine availability across different medical stores. The specific objectives of the system are as follows:

- **Real-Time Medicine Search:** To design a responsive system capable of instantly retrieving medicine details based on user input, such as medicine name or common symptoms like fever, cough, or headache.
- **Efficient Medicine Information Display:** To present key details such as medicine name, company, type, price, and availability in an organized tabular format, allowing users to easily compare options.
- **Dynamic Inventory Management:** To provide registered medical stores with a secure portal to add, update, or delete their medicine records, ensuring that the displayed information remains accurate and up to date.
- **Admin Management Dashboard:** To enable administrators to manage medical stores, view store details, and oversee the registration and maintenance of store information.
- **Scalable and Secure Backend:** To implement a robust backend using Flask connected to MongoDB, ensuring secure data storage, efficient retrieval, and smooth communication with the frontend.

1.6 Report Organization

This report details the complete lifecycle of the **“Medicine Finder”** project.

- **Chapter 2** provides a literature survey of existing medicine search and e-

pharmacy solutions, identifies their limitations, and formulates the problem statement that motivated this work.

- **Chapter 3** gives a high-level overview of the proposed Medicine Finder system, explaining its core architecture, modules, and workflow.
- **Chapter 4** presents a detailed analysis of the system's requirements (functional, non-functional, and design-related) and outlines the complete system architecture, including database structure and data flow diagrams.
- **Chapter 5** describes the implementation methodology, backend integration, frontend interface, and showcases the running application through various screenshots illustrating the core functionalities.
- **Chapter 6** discusses the results of the project, evaluating the system's performance, accuracy, and efficiency in meeting user needs across different use cases.
- **Chapter 7** concludes the report by summarizing the key outcomes and technical learnings, and proposes future enhancements such as GPS-based navigation, real-time stock synchronization, and AI-driven medicine recommendations.

CHAPTER 2: LITERATURE SURVEY

2.1 Survey of the Existing Models/Work

The concept of using technology to simplify medicine accessibility and pharmacy management is not new. The current healthcare and pharmaceutical industries already feature several digital tools that indicate a clear shift from manual search and physical availability checks toward intelligent, real-time, and location-based medicine discovery systems.

- **Commercial Platforms:** Platforms such as 1mg, NetMeds, and PharmEasy are leading examples in the digital pharmacy ecosystem. These services allow users to order medicines online, view dosage information, and even consult healthcare professionals virtually. However, their core focus remains on home delivery, which introduces time delays and fails to address immediate medicine requirements. These tools validate the demand for systems that go beyond delivery and enable direct, local access to medicines.
- **Traditional Paradigms:** The most common existing approach, used in local pharmacy search systems, relies on static keyword-based lookup or manual browsing of store inventories. This paradigm is often inefficient because it depends on exact text matches and cannot interpret user intent or context, such as searching

by symptom or related conditions. As a result, users must manually explore multiple stores or websites to find what they need, highlighting the need for a smarter, context-aware, and real-time solution.

2.2 Gaps Identified in the Survey

The primary gap lies in the limitations of existing medicine search and e-pharmacy platforms. While current systems provide convenience through online ordering, they remain “**fundamentally limited**” in addressing the immediate and location-specific needs of users seeking urgent medical supplies.

- **Lack of Real-Time Availability:** Most existing e-pharmacy applications focus on order placement and delivery logistics rather than real-time stock visibility. Users often discover only after checkout that a medicine is unavailable or delayed, defeating the purpose of immediate accessibility.
- **Absence of Location Awareness:** Traditional systems are not optimized for proximity-based discovery. They fail to leverage geolocation data to suggest nearby pharmacies, forcing users to manually search multiple platforms or physically visit stores.
- **Limited Interactivity and Comparison:** Existing platforms lack intelligent mechanisms for comparing medicines across parameters like brand, price, and form. They simply list products without helping users make context-aware, data-driven choices.

2.3 Problem Statement

The central problem is that users often face significant difficulty in finding specific medicines quickly and reliably, especially during emergencies. Existing solutions for locating medicines are inadequate. Most e-pharmacy systems focus solely on online ordering and delivery, which introduces time delays, while local search engines lack real-time data about medicine stock and availability. Additionally, current platforms do not provide intelligent filtering or comparison features, forcing users to manually explore multiple sources before finding what they need.

Therefore, the problem is to design and build an intelligent system that can:

1. Aggregate and unify data from multiple registered medical stores into a single, searchable platform.
2. Move beyond simple keyword-based search to enable intelligent, context-aware discovery based on medicine names, types, or symptoms.

3. Provide users with real-time details about availability, pricing, and location of nearby pharmacies.
4. Offer an interactive, user-friendly interface that allows comparison between multiple medicines, ensuring accurate, transparent, and efficient access to essential healthcare resources.

CHAPTER 3: OVERVIEW OF THE PROPOSED SYSTEM

The proposed “**Medicine Finder**” system is a sophisticated, full-stack web application designed to function as an intelligent, real-time medicine discovery and comparison platform. It follows a **three-tier architecture** to seamlessly integrate pharmacy data, user interaction, and database management into a unified, efficient, and accessible system.

The system architecture is designed for **modularity, scalability, and reliability**. At a high level, it consists of four primary components:

1. **React.js Frontend:** A modern, responsive web interface built with React and React-Bootstrap. It serves as the user’s primary point of interaction, allowing them to search for medicines by name or symptom, apply filters, and compare up to three medicines simultaneously. The use of React-Bootstrap ensures a clean, consistent design and efficient layout management across devices.
2. **Node.js + Express Backend:** A robust RESTful API that serves as the “core engine” of the system. It manages all application logic, handles search and comparison requests, and ensures seamless data exchange between the frontend and the database.
3. **MongoDB Database:** A scalable NoSQL database that stores structured information about medicines, pharmacy details, availability, and user records. It supports quick query execution and efficient real-time data updates from registered medical stores.
4. **Pharmacy Admin Module:** A secure, login-based interface that allows authorized medical stores and administrators to register, upload, and manage their medicine inventories, ensuring that all information displayed to users is accurate and up to date.

This architecture is implemented as a **comprehensive four-layer framework**, ensuring both functional efficiency and user accessibility:

1. **Layer 1: Data Entry and Processing:** Collects and validates medicine data submitted by pharmacies through the admin interface, ensuring consistency in fields such as name, type, manufacturer, price, and quantity.
2. **Layer 2: Data Storage and Indexing:** Maintains structured datasets within MongoDB, supporting indexing for faster search and retrieval. Each medicine entry is associated with metadata like location and store ID for optimized performance.
3. **Layer 3: Query and Retrieval Engine:** Handles user input by processing search queries (by name or symptom) and retrieving relevant records from the database. The system intelligently filters results and ranks them by relevance and proximity.
4. **Layer 4: User Interaction and Visualization:** Displays the final search results in an intuitive and organized format, enabling users to view store details, filter by medicine type, and compare multiple options. This layer also integrates map-based navigation and contact details for convenience.

This end-to-end system provides a complete blueprint for transforming the conventional, manual process of finding medicines into a **dynamic, intelligent, and real-time assistance platform** that empowers users with immediate, trustworthy, and actionable information.

CHAPTER 4: REQUIREMENTS ANALYSIS AND DESIGN

4.1 Requirements Analysis

4.1.1. Functional Requirements

FR1: User-Friendly Search: The system must allow users to search for medicines based on either the medicine name or related symptoms, ensuring accessibility for users who may not know the exact drug name.

- **FR2: Location-Based Results:** The system must use location data to identify and display pharmacies nearest to the user, helping them access medicines quickly and conveniently.

- **FR3: Medicine Filtering:** The system must allow users to filter medicines based on their type (e.g., tablet, syrup, injection, ointment, capsule) to refine search results according to preferences.

- **FR4: Medicine Comparison:** The system must enable users to compare up to three medicines simultaneously based on key parameters like manufacturer, price, type, and store details.
- **FR5: Pharmacy Registration:** The system must allow administrators to register medical stores and provide each store with a secure login credential for managing their inventory.
- **FR6: Medicine Management:** Registered pharmacies must be able to upload, edit, and delete medicine information such as name, quantity, expiry date, and price through a dedicated dashboard.
- **FR7: Real-Time Availability:** The system must reflect current stock levels in real time, ensuring that users only view medicines that are actually available in stores.
- **FR8: Search Optimization:** The system must implement optimized database queries to ensure fast and accurate retrieval of medicine and store data from the backend.
- **FR9: Responsive Frontend:** The web interface, built using **React and React-Bootstrap**, must be fully responsive and provide seamless access on desktops, tablets, and smartphones.
- **FR10: Secure Authentication:** The system must implement a secure authentication mechanism for administrators and pharmacy owners to prevent unauthorized access and maintain data integrity.

4.1.2. Product Perspective

The product is an **interactive medicine discovery platform** and **intelligent pharmacy locator** for users. It is a full-stack web application that transforms a user's manual and time-consuming process of finding medicines into an active, efficient, and real-time search experience. It serves as a **personal, location-aware, and reliable assistant** that helps individuals quickly identify nearby pharmacies with the required medicines in stock. It also supports small and local pharmacies by giving them greater digital visibility and helping them reach potential customers more effectively. It is not a general-purpose e-commerce or delivery service but a **domain-specific healthcare information system** that operates exclusively on verified pharmacy and medicine data provided by registered stores.

4.1.3. Product Features

The product is an “**AI-powered medicine discovery assistant**” and “**intelligent pharmacy locator**” for users. It is a full-stack web application that transforms a user's

manual search for medicines into an automated, intelligent, and real-time discovery experience. It serves as a **personal, fast, and reliable healthcare assistant** that helps individuals find nearby pharmacies, check medicine availability, and compare multiple options. It is not a general-purpose e-commerce platform but a **domain-specific medical information engine** that operates exclusively over verified pharmacy data and inventories.

- **Symptom-Based Search:** A smart search feature that allows users to find medicines not only by name but also by the symptoms they are experiencing (e.g., fever, cough, headache).
- **Medicine Filtering:** An advanced filtering system that enables users to sort medicines by type, such as tablets, capsules, syrups, or ointments, improving precision and convenience.
- **Medicine Comparison:** A dedicated interface where users can compare up to three medicines simultaneously based on their price, type, brand, and manufacturer details.
- **Pharmacy Dashboard:** A secure and user-friendly dashboard that allows registered pharmacies to upload, update, and manage their available stock in real time.
- **Admin Management:** An administrative interface where the system admin can register new medical stores, manage user permissions, and monitor the accuracy of store data.
- **Responsive Design:** A fully responsive frontend developed using **React and React-Bootstrap**, ensuring smooth operation across desktops, tablets, and mobile devices.
- **Real-Time Data Synchronization:** Automatic updates to reflect changes in stock availability and newly added medicines without requiring a full page reload, maintaining data freshness and reliability.

4.1.4. User Characteristics

The primary user is the **general public**, including individuals living in both urban and rural areas who need quick access to medicines. These users often face challenges when searching for specific medicines across multiple pharmacies, especially during emergencies. They require a simple, reliable, and real-time tool that helps them locate nearby medical stores, check medicine availability, and compare options efficiently. Additionally, the system also serves **pharmacy owners and administrators**, who can use it to register their stores, manage inventories, and gain visibility among potential customers.

4.1.5. Assumption & Dependencies

- **Assumptions:**

- It is assumed that pharmacies registering on the platform will provide accurate, up-to-date, and verified information about their medicine inventories, including name, type, price, and stock availability.
- It is assumed that users will have access to a stable internet connection and location services enabled on their device to ensure accurate proximity-based search results.
- It is assumed that all registered medical stores will adhere to legal and ethical regulations concerning the sale of medicines and maintain valid pharmacy licenses.

- **Dependencies:**

- **External APIs:** The system is dependent on external APIs and services for key functionalities:
 1. **Geolocation Service:** Google Maps or OpenStreetMap API for retrieving and displaying nearby pharmacies based on user location.
 2. **Email/Authentication Service:** Firebase Authentication or JWT for secure login and registration of pharmacy accounts.
 3. **Cloud Hosting:** A web hosting service such as Render, Vercel, or AWS for reliable deployment and uptime management of both frontend and backend systems.
- **Open-Source Libraries and Frameworks:** The system relies on a combination of open-source technologies and frameworks, including:
 - Node.js and Express.js (Backend).
 - React.js and React-Bootstrap (Frontend).
 - MongoDB and Mongoose (Database and ORM).
 - Axios (API communication).
 - JSON Web Tokens (JWT) (Authentication).
 - Bootstrap Icons and React Router DOM (UI and navigation)

4.1.6. Domain Requirements

The system operates within the **healthcare and pharmaceutical** domain, which demands accuracy, reliability, and compliance.

- **Data Accuracy:** All medicine details, including name, type, and availability, must be verified and regularly updated by registered pharmacies.

- **Regulatory Compliance:** Only licensed pharmacies are permitted to register and display stock information, ensuring adherence to healthcare standards.
- **User Safety:** The system must provide clear pharmacy details such as name, address, and contact information to ensure trust and transparency.

4.1.7. Non-Functional Requirements

- **Performance:** The system must provide real-time or near-real-time search results when users query for medicines.
 - **Retrieval Speed:** The database queries must execute quickly, ensuring that search results and store details are displayed instantly. This is achieved by optimizing MongoDB indexes and query operations.
 - **Backend Throughput:** The backend must efficiently handle multiple user requests simultaneously, achieved through asynchronous processing in Node.js and optimized API endpoints.
- **Scalability:** The system must be capable of scaling horizontally to accommodate more users, pharmacies, and data as adoption grows.
 - **Containerization:** The backend is containerized using Docker, enabling smooth deployment and replication across environments.
 - **Modular Architecture:** The system's modular design allows independent scaling of components such as authentication, database, and search services.
- **Reliability:** The system must remain stable even under heavy load or API failure. Redundant services and efficient error handling ensure uninterrupted access and consistent data availability.
- **Usability:** The web interface must be clean, responsive, and intuitive. The **React-Bootstrap** frontend ensures smooth navigation and accessibility across desktops, tablets, and mobile devices.
- **Maintainability:** The architecture must remain modular and well-documented, allowing developers to update, optimize, and maintain individual components—such as search logic or admin management—without disrupting the entire system.

4.2 System Modelling

The system's processes and data flow are modeled through several diagrams that collectively describe the architecture, interactions, and flow of data within the **Medicine Finder** platform.

1. High-Level System Architecture:

The *System Architecture Overview* diagram illustrates the overall component interaction. It shows the **Frontend (React-Bootstrap web interface)** connecting to the **Backend (Node.js and Express server)**, which in turn communicates with

the **MongoDB database** for storing and retrieving medicine and pharmacy details. The architecture ensures a clear separation of concerns, allowing for modular development and independent scaling of each layer.

2. **Medicine Search Process Flow (DFD):**

The *Medicine Search Workflow* serves as a detailed Data Flow Diagram for the core functionality:

- **(Process 1) User Input:** The user enters the name of a medicine or a related symptom into the search bar.
- **(Process 2) Query Handling:** The backend receives the query and processes it through optimized database queries.
- **(Process 3) Data Retrieval:** Relevant medicine entries are fetched from MongoDB, along with corresponding pharmacy details.
- **(Process 4) Filtering and Sorting:** The system filters results based on type, availability, and location, then ranks them by proximity.
- **(Process 5) Display Results:** The processed data is sent back to the frontend and displayed in a clean, interactive table for the user.

3. **Medicine Comparison Flow:**

The *Comparison Module* diagram represents the user flow when comparing up to three medicines. It includes stages such as **Selection of Medicines**, **Data Fetching**, **Attribute Comparison (price, type, manufacturer)**, and **Result Visualization** through a dynamic table. This ensures users can make informed decisions efficiently.

4. **Database Architecture:**

The *Database Schema Overview* diagram illustrates the structure of the **MongoDB collections** used in the system. It includes collections such as **Medicines**, **Pharmacies**, and **Admins**, each linked through unique identifiers. The database supports fast queries and real-time updates, ensuring synchronization between pharmacy uploads and user search results.

4.3 Engineering Standard Requirements

● **Economic:**

The project involves moderate operational costs, mainly related to hosting and maintenance.

1. **Hosting Services:** The application's backend and frontend deployment on cloud platforms such as Render or Vercel incur basic monthly costs.
2. **Database and API Usage:** MongoDB Atlas and Google Maps API usage may generate minimal recurring charges depending on query volume and data storage.
 - To ensure sustainability, the system is designed with efficient query handling, caching mechanisms, and lightweight API calls to minimize costs while maintaining performance.

● **Societal Need:**

The project addresses a clear societal need for **accessible and real-time healthcare information**. It benefits users by enabling them to locate nearby pharmacies during

emergencies, and it also helps **small, local pharmacies gain online visibility**, promoting equitable access to healthcare resources in both urban and rural areas.

- **Ethical:**

The primary ethical consideration is ensuring **accuracy and trustworthiness** of displayed medicine data. All pharmacy entries are verified during registration, and the system restricts unlicensed sellers from uploading or modifying data. No misleading or promotional content is permitted, ensuring user safety and reliability of information.

- **Transparency (Verifiability):**

Users are provided with **clear pharmacy details**, including address, contact information, and store credentials. This transparency allows them to verify authenticity before visiting or purchasing medicines, thereby increasing trust in the platform.

- **Sustainability:**

The project's long-term sustainability is ensured by using **open-source technologies** such as React, Node.js, and MongoDB, which minimize costs and promote scalability. The modular architecture supports future upgrades, including AI-based recommendations and automated inventory updates, without major redevelopment expenses.

4.4 System Requirements

4.4.1 Hardware Requirements

The **Medicine Finder** system is a **web-based application** that can be deployed on the cloud or hosted locally. Therefore, hardware requirements are defined primarily for the deployment environment rather than for individual users.

- **Client-Side:**

Any modern web browser running on a desktop, laptop, tablet, or smartphone with a stable internet connection. The system is compatible with Chrome, Edge, Firefox, and Safari.

- **Server-Side (Deployment Environment):**

- **Application Server:** A standard virtual machine instance (e.g., Render, AWS EC2, or Vercel) capable of running the Node.js backend and React-Bootstrap frontend.
- **Database Server:** A MongoDB Atlas or self-hosted MongoDB instance to store medicine, pharmacy, and user data securely and efficiently.
- **Storage Requirements:** Basic SSD-based storage for database operations and user uploads, with capacity scaling as the number of registered pharmacies increases.

4.4.2. Software Requirements

Core Technology Stack:

Layer	Technologies	Description
Frontend	React.js, React-Bootstrap, HTML, CSS, JavaScript	Modern, mobile-responsive web interface with interactive search and filtering.
Backend	Node.js, Express.js	RESTful API for handling user queries and processing pharmacy data.
Database	MongoDB, Mongoose ORM	NoSQL database designed for flexible, real-time medicine and pharmacy record management.
Utilities	Axios, dotenv	Facilitates efficient API communication and environment configuration.

Detailed System Components:

System Stage	Recommended Technology/Service	Rationale
Frontend Development	React.js, React-Bootstrap	Provides a responsive, user-friendly interface for seamless navigation and real-time updates.
Backend Development	Node.js with Express.js	Handles all application logic, routing, and API communication efficiently.
Database Management	MongoDB with Mongoose	Stores medicine details, pharmacy information, and user data in a scalable NoSQL format.
Comparison Engine	Custom-built Comparison Module	Allows users to compare up to three medicines by price, brand, and type.

4.5 System Design

4.5.1. System Architecture

The system is designed using a modular, four-layer architecture, which provides a complete, end-to-end framework for efficient medicine search, pharmacy registration, and real-time data management.

- Layer 1: User Interaction and Input Handling:

This is the system's entry point, where users interact with the application through a responsive React-Bootstrap interface. It allows users to search for medicines by name or symptom, filter results based on type, and compare up to three medicines at a time. Pharmacy owners and the admin can also log in through this layer to upload or update medicine details.

- **Layer 2: Data Management and Validation:**

This layer handles the collection and organization of data from different users and pharmacies. It ensures that all uploaded information, including medicine names, quantities, prices, and expiry dates, is validated and stored in a standardized format. This prevents errors such as duplicate entries or missing data and maintains the integrity of the database.

- **Layer 3: Database and Storage:**

This layer serves as the system's central repository, using MongoDB to store medicine details, pharmacy profiles, and admin records. Indexed fields like medicine name, pharmacy ID, and stock status allow fast and efficient data retrieval. The database is designed to handle real-time updates, so any changes made by a pharmacy are immediately reflected for users.

- **Layer 4: Processing and Result Display:**

This is the interactive, output-oriented layer that manages all backend logic. When a user performs a search, the backend (built with Node.js and Express) processes the query, fetches relevant data from the database, and returns accurate results. The results are displayed in a neatly formatted table showing the medicine details, available stores, and prices. This layer also supports sorting, filtering, and comparison features, ensuring a smooth and informative user experience.

4.5.2 Detailed Design

The system architecture is implemented through a series of distinct, interacting modules that together form the complete **Medicine Finder** platform. Each module plays a specific role in data processing, storage, and user interaction to ensure smooth functionality and reliable results.

- **Module 1: User Interaction Module (Frontend/)**

- This module represents the **Layer 1 (User Interface)** of the system. It is built using **React.js** and **React-Bootstrap** to provide a responsive and user-friendly experience.
- **Home Page:** Allows users to search for medicines by name or symptom, and displays the results in a clean, tabular format.
- **Comparison Page:** Enables users to compare up to three medicines at a time based on price, type, and manufacturer.
- **Admin and Pharmacy Login:** Provides access for registered users to manage their store or for the admin to register new pharmacies.
- **Routing:** Implemented using React Router DOM to allow smooth transitions between the home, search, and admin interfaces.

- **Module 2: Data Management Module (Backend/)**

- This module implements **Layer 2 (Data Handling)** and is built using **Node.js** and **Express.js**. It serves as the communication layer between the frontend and the database.
- Handles all CRUD (Create, Read, Update, Delete) operations for medicines and pharmacy data.

- Provides RESTful API endpoints for medicine search, pharmacy registration, and inventory updates.
- Implements input validation and error handling to ensure the integrity of the data received from the frontend.

- **Module 3: Database Module (Database/)**

- This module represents **Layer 3 (Storage and Indexing)** and uses **MongoDB** as the database system.
- It stores structured data such as medicine details, pharmacy profiles, admin credentials, and stock availability.
- Indexing is implemented on frequently searched fields (e.g., medicine name, type) to optimize query speed.
- Each pharmacy has its own medicine collection, ensuring data isolation and accurate record management.

- **Module 4: Medicine Comparison and Filtering Module (Comparison/)**

- This module is responsible for enabling **real-time filtering and comparison** features.
- It handles user requests to compare medicines and displays results based on manufacturer, type, and price.
- Filtering logic is implemented to categorize medicines by form (e.g., tablet, syrup, injection) and availability.
- Ensures that users receive accurate and meaningful results for quick decision-making.

- **Module 5: Pharmacy Management Module (Pharmacy/)**

- This module provides an interface for pharmacy owners to upload and manage their medicine inventory.
- Each registered store can add, update, or delete medicines, ensuring the system always reflects real-time availability.
- Validation checks ensure that all entries contain necessary details like name, price, expiry date, and stock quantity.

- **Module 6: Admin Management Module (Admin/)**

- This module allows the system administrator to register new pharmacies, manage existing ones, and oversee platform operations.
- It provides admin-level access control for maintaining data consistency and monitoring store authenticity.
- Admins can verify pharmacy credentials and ensure that only licensed stores remain active on the platform.

- **Module 7: Utility Module (Utils/)**

- Contains helper scripts and middleware functions to streamline backend operations.
- Includes configuration files, API connection handlers, and logging utilities for monitoring server performance.
- Manages environment variables (via **dotenv**) and API responses for consistent backend performance and security.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Methodology

The core methodology of the **Medicine Finder** project is based on a structured, multi-stage workflow that enables users to search, compare, and access real-time medicine and pharmacy information. The implementation of this methodology follows a precise six-stage process, forming the system's main operational pipeline:

- 1. User Input and Search:**
Users enter a medicine name or related symptom through the React-Bootstrap frontend. The query is sent to the backend API, which validates the input and prepares it for database lookup.
- 2. Data Retrieval:**
The backend (Node.js + Express.js) receives the query and searches the MongoDB database for matching medicines. It retrieves relevant records along with corresponding pharmacy details such as store name, address, and contact information.
- 3. Filtering and Sorting:**
The system applies filters based on medicine type (tablet, syrup, capsule, etc.), brand, or price range. The results are then sorted according to availability and proximity, ensuring users can quickly find suitable options.
- 4. Medicine Comparison:**
Users can select up to three medicines to compare side-by-side. The system dynamically fetches detailed information — including brand, manufacturer, price, and form — and displays it in a tabular format for clear visualization.
- 5. Pharmacy Management:**
Registered pharmacies can log in through their dedicated dashboard to add, update, or delete medicines. These updates are instantly reflected in the database, ensuring that the user-facing data remains accurate and current.
- 6. Result Display and Interaction:**
The frontend presents all results in an organized, responsive interface. Users can view detailed medicine information, check store availability, and directly contact pharmacies if needed. The interface is optimized for speed and usability across both desktop and mobile platforms.

This entire process, from search input to result display, is automated and managed through the backend's RESTful APIs, ensuring smooth, real-time communication between the user interface and the database.

5.2 Dataset Description

The “dataset” for the **Medicine Finder** project is not a static or predefined collection of data. Instead, it is a **dynamic and evolving dataset** that depends on the information provided by registered pharmacies and the search queries made by users. Each data entry

in the system reflects real-world, continuously updating information about medicines and stores.

The system is designed to manage this **structured yet frequently changing dataset**, which consists of:

- **Pharmacy Records:** Data submitted by medical store owners through the pharmacy dashboard, including store name, address, contact details, and registration ID.
- **Medicine Details:** Entries uploaded by pharmacies containing the medicine name, type (tablet, syrup, capsule, injection), manufacturer, price, quantity, and expiry date.
- **User Search Queries:** Real-time user inputs that form a temporary dataset used for filtering and generating relevant search results.

5.3 Implementation Setup and Screenshots

The system was developed and tested locally to ensure a stable and responsive user experience. The **backend (Node.js with Express)** was executed using the command: `nodemon index.js` and the **frontend (React with React-Bootstrap)** was run using: `npm start`

The project follows a modular full-stack architecture, integrating a **MongoDB database**, **Express.js API**, and a **React-based frontend interface**.

The following screenshots demonstrate the working prototype of the **Medicine Finder** application:

Figure 5.1: Home Page (Medicine Search Interface)

This screenshot shows the main homepage of the application. The user is presented with a simple and elegant search bar to find medicines easily. Below the search bar, there are quick-access symptom buttons (e.g., Fever, Headache, Cold, Cough, etc.) that allow users to instantly search for medicines based on common health conditions.

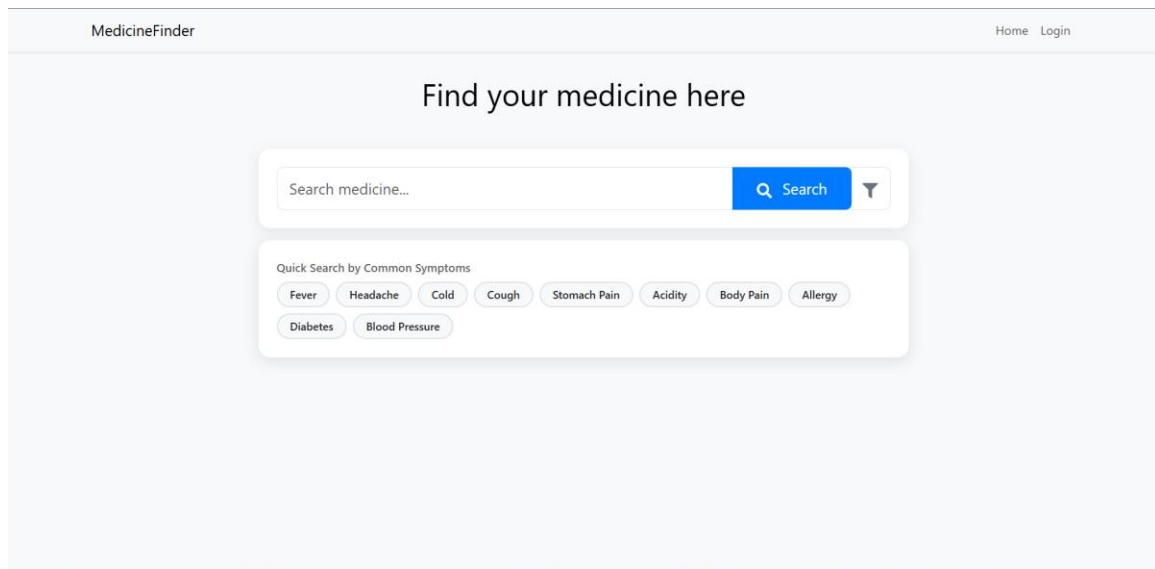


Figure 5.2: Search Results Display (Detailed Medicine View)

This screenshot demonstrates the system’s functionality after the user searches for a symptom such as “*Fever*.” The interface displays a structured and easy-to-read list of medicines available for that condition. Each row provides key information including the **medicine name**, **company**, **store name**, **price**, **type**, **contact number**, and **address** of the pharmacy.

The results are sorted by default (e.g., alphabetically by name) but can be adjusted using the “**Sort by**” dropdown menu. The page also includes features like **store rating** and **direct contact options**, allowing users to call or reach the pharmacy conveniently.

This view ensures that users can quickly compare medicine options and choose based on availability, location, and cost—making the process efficient and user-friendly while promoting visibility for local pharmacies.

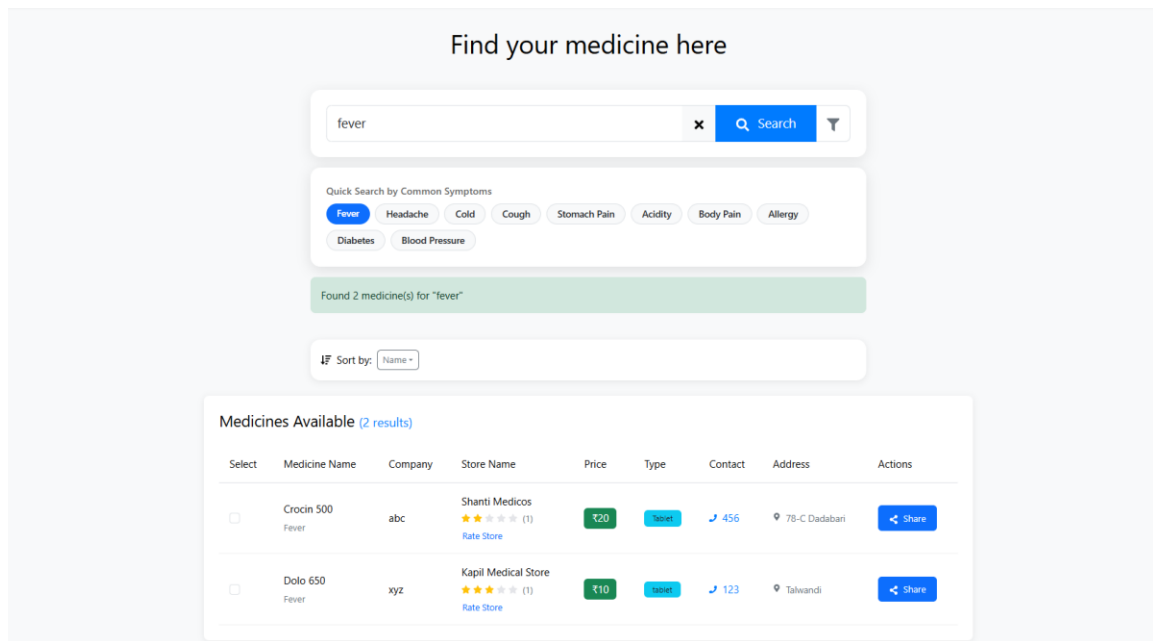


Figure 5.3: Admin Dashboard (Profile View)

This screenshot displays the **Admin Dashboard** interface of the Medicine Finder system. The interface welcomes the admin and provides a clear view of the **Admin Profile**, which includes key details such as **Name**, **Address**, **Contact Number**, and **Email ID**.

The navigation bar at the top provides access to major admin functionalities like **Add Admin**, **Register Medical Store**, **Manage Stores**, **Edit Profile**, and **Logout**. This dashboard enables the admin to efficiently manage the registered medical stores and oversee system operations in a clean, user-friendly interface.

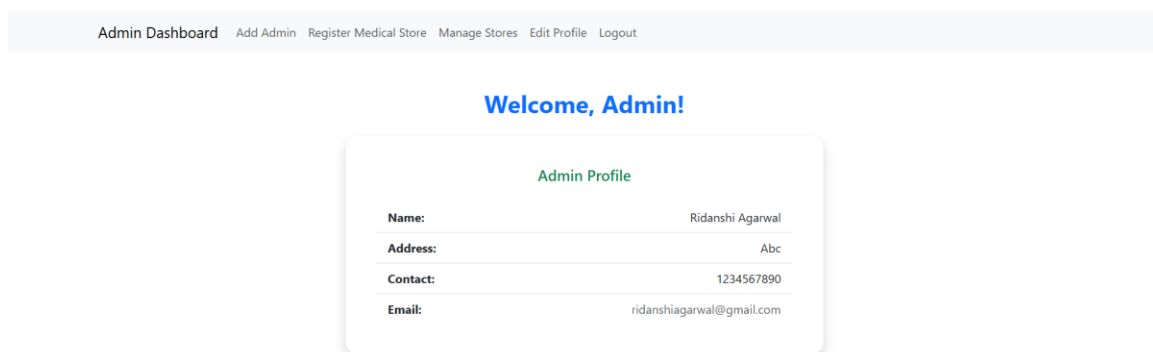
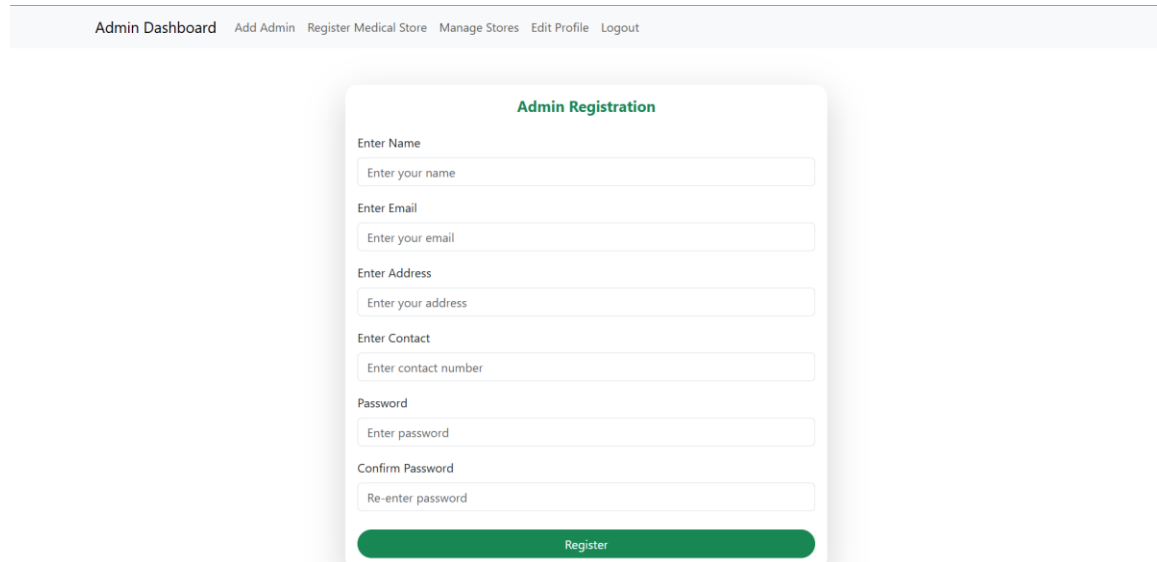


Figure 5.4: Admin Registration Page

This screenshot shows the **Admin Registration Page** of the Medicine Finder system. It allows a new admin to register by entering essential details such as **Name**, **Email**, **Address**, **Contact Number**, and **Password** (with confirmation).

The page features a clean and minimalistic design with a focus on ease of use. Once the form is filled out, the admin can click the **“Register”** button to successfully create an account. This functionality ensures secure onboarding of new administrators who can later manage pharmacy registrations and oversee system data.



The screenshot displays the Admin Registration Page of the Medicine Finder system. At the top, a navigation bar includes links: Admin Dashboard, Add Admin, Register Medical Store, Manage Stores, Edit Profile, and Logout. The main content area features a form titled "Admin Registration" with the following fields: Enter Name (placeholder: Enter your name), Enter Email (placeholder: Enter your email), Enter Address (placeholder: Enter your address), Enter Contact (placeholder: Enter contact number), Password (placeholder: Enter password), and Confirm Password (placeholder: Re-enter password). A green "Register" button is positioned at the bottom of the form.

Figure 5.5: Medical Store Registration Page

This screenshot shows the **Medical Store Registration Page**, which allows the admin to register new pharmacies within the Medicine Finder system. The form collects detailed information such as **Store Name**, **Owner Name**, **Address**, **Contact Number**, **License Number**, **Email**, and **Password**, along with an option to upload a **License Document** for verification.

Once all details are filled in, the admin can click the **“Register”** button to successfully onboard a medical store into the system. This feature ensures that only **verified and licensed pharmacies** are added, promoting authenticity and reliability across the platform.

Admin DashboardAdd AdminRegister Medical StoreManage StoresEdit ProfileLogout

Medical Store Registration

Store Name

Enter store name

Owner

Enter owner's name

Address

Enter address

Contact

Enter contact number

License Number

Enter license number

Email

Enter email

Password

Enter password

Confirm Password

Confirm password

License Document

Choose File

No file chosen

Register

Figure 5.6: Manage Registered Medical Stores Page
This screenshot displays the **Manage Registered Medical Stores Page**, where the admin can view, edit, or delete details of all pharmacies registered in the system. Each entry lists key information such as **Store Name**, **Owner**, **Address**, **Contact Number**, **License Number**, and **Email**.

The **Edit** and **Delete** buttons under the *Actions* column allow the admin to update store information or remove inactive or unverified medical stores, ensuring that the Medicine Finder database remains accurate and up to date.

Admin DashboardAdd AdminRegister Medical StoreManage StoresEdit ProfileLogout

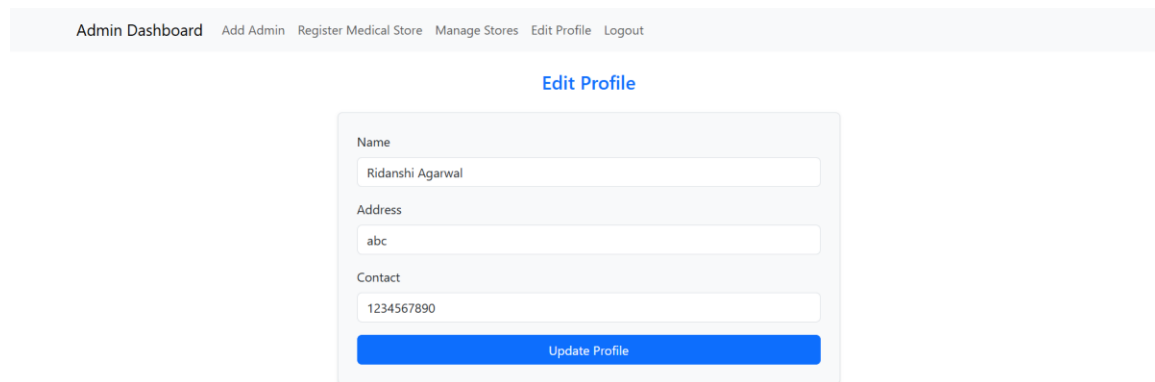
Registered Medical Stores

Store Name	Owner	Address	Contact	License #	Email	Actions	
Kapil Medical Store	Kapil Sharma	Talwandi	123	23	k@gmail.com	Edit	Delete
Shanti Medicos	Harish Lal	78-C Dadabari	456	45	h@gmail.com	Edit	Delete

Figure 5.7: Edit Profile Page

This screenshot shows the **Edit Profile Page** of the admin panel. It allows the admin to update their personal details, including **Name**, **Address**, and **Contact Number**.

The interface provides editable input fields pre-filled with existing information, and an **“Update Profile”** button to save any modifications. This feature ensures that administrators can maintain accurate and up-to-date contact and address information within the system.



The screenshot displays the 'Edit Profile' page within an admin dashboard. At the top, a navigation bar contains links: 'Admin Dashboard', 'Add Admin', 'Register Medical Store', 'Manage Stores', 'Edit Profile', and 'Logout'. Below this, the page title 'Edit Profile' is centered. The main content area features a form with three input fields: 'Name' (pre-filled with 'Ridanshi Agarwal'), 'Address' (pre-filled with 'abc'), and 'Contact' (pre-filled with '1234567890'). A blue 'Update Profile' button is positioned at the bottom of the form.

Figure 5.8: Medical Dashboard – Profile Page

This screenshot displays the **Medical Dashboard Profile Page**, which provides a detailed overview of a registered medical store’s information.

The page shows the store’s **name**, **owner details**, **address**, **contact number**, **email**, and **license number** in a neatly formatted profile card. The dashboard greets the user with a welcome message (“Welcome, Medical!”) and provides navigation options at the top, such as **Add Medicine**, **Medicine List**, **Edit Profile**, and **Logout**. This interface helps medical store owners manage their profiles and operations efficiently.

Medical DashboardAdd MedicineMedicine ListEdit ProfileLogout

Welcome, Medical!

Medical Profile

Store Name:	Shanti Medicos
Owner:	Harish Lal
Address:	78-C Dadabari
Contact:	456
Email:	h@gmail.com
License:	45

Figure 5.9: Medicine Registration Page

This screenshot illustrates the **Medicine Registration Page** within the **Medical Dashboard**. It allows a medical store to add new medicines to the system by entering key details such as **medicine name**, **company**, **license number**, **description**, **unit price**, and **medicine type**.

The form ensures structured data entry for maintaining accurate medicine records. Once the fields are filled, the user can click the **“Register Medicine”** button to save the details to the database. The top navigation bar provides quick access to other dashboard sections like **Add Medicine**, **Medicine List**, **Edit Profile**, and **Logout**.

Medical DashboardAdd MedicineMedicine ListEdit ProfileLogout

Medicine Registration

Medicine Name

Enter medicine name

Company

Enter company name

License

Enter license

Medicine Description

Enter description

Unit Price

Enter unit price

Medicine Type

Enter medicine type

Register Medicine

Figure 5.10: Medicine List Page

This screenshot shows the **Medicine List Page** under the **Medical Dashboard**, which displays all medicines registered by the medical store. The data is presented in a tabular format containing details such as **Medicine Name**, **Company**, **License Number**, **Description**, **Unit Price**, and **Type**.

Each row also includes **Edit** and **Delete** buttons, allowing the user to modify or remove medicine entries easily. This interface provides a clear and organized view for managing medicine inventory within the application.

Medical DashboardAdd MedicineMedicine ListEdit ProfileLogout

All Medicine Data

Medicine Name	Company	License Number	Description	Unit Price	Type	Actions	
Crocin 500	abc	abc	Fever	20	Tablet	Edit	Delete

Figure 5.11: Edit Medical Profile Page

This screenshot displays the **Edit Medical Profile Page** from the **Medical Dashboard**. The interface allows the medical store owner to update essential details such as **Store Name**, **Owner Name**, **Address**, **Contact**, and **License Number**.

After making the necessary changes, the user can click the **Update Profile** button to save the updated information. This feature ensures that medical store details remain accurate and up to date within the system.

The screenshot shows the 'Edit Medical Profile' page. At the top, there is a navigation bar with links: 'Medical Dashboard', 'Add Medicine', 'Medicine List', 'Edit Profile', and 'Logout'. Below the navigation bar, the title 'Edit Medical Profile' is displayed in blue. The main content area contains a form with the following fields:

- Store Name:** Shanti Medicos
- Owner Name:** Harish Lal
- Address:** 78-C Dadabari
- Contact:** 456
- License Number:** 45

At the bottom of the form is a blue button labeled 'Update Profile'.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Research Finding

The primary finding of this project is that the **Medicine Finder System** is an efficient and user-friendly solution for locating medicines and their availability across registered medical stores. The implementation confirms that this system effectively addresses the two main challenges identified in the problem statement:

- Overcomes the difficulty of manual medicine search:**
By allowing users to search medicines through **keywords or common symptoms**, the system simplifies the process of finding required medicines. The integrated search and filtering mechanism quickly displays all relevant medicines

along with details such as **store name, price, contact, and address**, eliminating the need to visit multiple pharmacies physically.

2. **Ensures data accuracy and easy management:**

The inclusion of **admin and medical dashboards** ensures that all information — such as store details, medicine data, and user records — remains accurate and regularly updated. The **admin verification process** and structured database prevent duplicate or false entries, ensuring the reliability and trustworthiness of the results shown to users.

6.2 Result Analysis

The result of the project is a fully functional, end-to-end **Medicine Finder Web Application**. The system successfully fulfills all the core objectives defined in section 1.5.

- The system efficiently helps users **search medicines by name or symptoms**, displaying detailed information such as **price, company, store name, contact, and address** in a clear and organized format.
- It provides a **quick search feature** for common symptoms (like fever, cough, or headache), improving accessibility and user convenience.
- The platform includes **separate dashboards for Admin and Medical Stores**, allowing seamless management of data — admins can add or approve stores, while medical stores can register and update medicine details.
- The web application features a **clean, responsive, and easy-to-navigate interface** built using modern frontend tools, ensuring a smooth experience for both users and pharmacy owners.
- The backend ensures **secure and structured data handling**, maintaining accuracy and consistency in all registered store and medicine records.
- The project successfully integrates **core functionalities like medicine registration, editing, deletion, and search filtering**, achieving a complete, working prototype that enhances medicine accessibility for users while supporting small pharmacies in reaching a wider audience.

6.3 Evaluation

While a formal, quantitative evaluation was not conducted, the effectiveness of the **Medicine Finder System** can be assessed qualitatively based on two key performance aspects observed during testing and implementation:

1. **Efficiency of Medicine Retrieval:**

The system demonstrates high efficiency in retrieving relevant medicine information based on user input. The search functionality accurately displays medicines related to entered keywords or selected symptoms, ensuring users can quickly access the required data. The organized table-based display enables easy readability and comparison, contributing to an effective and user-friendly experience.

2. **Accuracy of Data Management:**

The accuracy and reliability of the system are primarily dependent on the integrity of the data uploaded by medical stores. Since registered pharmacies can directly add or update their stock details through the store dashboard, the information presented to users remains current and trustworthy. The admin's ability to manage store registrations further ensures data authenticity and system reliability.

CHAPTER 7: CONCLUSION AND SCOPE FOR FUTURE WORK

7.1 Conclusion

This project successfully designed, developed, and implemented the “**Medicine Finder**”, an intelligent web-based application aimed at simplifying the process of locating and managing medicine availability across pharmacies. The system effectively addresses the challenges faced by individuals in finding essential medicines quickly and reliably, especially in areas with limited access to medical stores.

By building upon a robust full-stack architecture with a **Flask backend** and a **React-based frontend**, the project achieved its key objectives:

1. It enabled real-time medicine search based on name or symptoms, providing users with instant and relevant results.
2. It provided pharmacy owners with a secure, dynamic dashboard to upload and manage their stock details, ensuring up-to-date information for users.
3. It allowed administrators to register, verify, and oversee medical stores, maintaining data accuracy and credibility.

The project's major technical accomplishments include developing an organized data management system, implementing efficient search functionality, and creating a clean, user-friendly interface suitable for both end users and pharmacy owners. Overall, the **Medicine Finder** system serves as a practical and impactful digital solution that connects users to nearby pharmacies and helps small medical stores gain visibility in a competitive healthcare environment.

7.2 Scope for Future Work

The current implementation of the **Medicine Finder** system establishes a strong foundation for a reliable and user-friendly medicine discovery and pharmacy management platform. However, there are several promising directions for future enhancement that could make the system more intelligent, accessible, and impactful.

● 1. **Location-Based Store Discovery:**

- **Limitation:** The current system does not integrate real-time geolocation to automatically identify nearby medical stores.
 - **Future Scope:** By integrating GPS and map-based APIs (such as Google Maps), the system can automatically detect a user's location and display the nearest pharmacies with the required medicines, making searches faster and more relevant.
- **2. Medicine Comparison Module:**
 - **Limitation:** At present, users can view medicine details but cannot directly compare multiple medicines side by side.
 - **Future Scope:** A comparison feature can be introduced to allow users to evaluate up to three medicines simultaneously based on parameters such as price, type, and manufacturer, helping them make more informed purchasing decisions.
- **3. Medicine Availability Notifications:**
 - **Limitation:** Users currently need to check the website manually for availability updates.
 - **Future Scope:** A notification system can be added to alert users when a previously unavailable medicine becomes available in nearby stores, improving user engagement and accessibility.
- **4. AI-Based Medicine Recommendations:**
 - **Limitation:** The system relies solely on user input for medicine search.
 - **Future Scope:** Future versions can incorporate AI-driven symptom analysis to suggest possible medicines based on user-entered symptoms, further enhancing usability and personalization.
- **5. Mobile Application Development:**
 - **Limitation:** The current system operates as a web-based platform.
 - **Future Scope:** Developing a companion mobile app for Android and iOS would increase accessibility, allowing users to locate medicines and pharmacies conveniently on the go.

REFERENCES

1. Medicine Finder Web App – Concept and Design Inspiration, accessed on October 28, 2025, <https://www.geeksforgeeks.org/web-development-projects/>
2. Flask Documentation – The Python Microframework for Web Applications, accessed on October 28, 2025, <https://flask.palletsprojects.com/>
3. MongoDB Official Documentation – NoSQL Database for Modern Applications, accessed on October 28, 2025, <https://www.mongodb.com/docs/>

4. React Bootstrap Documentation – Build Responsive Interfaces Faster, accessed on October 28, 2025, <https://react-bootstrap.github.io/>
5. Node.js vs Flask: Backend Comparison for Scalable Web Apps – Medium, accessed on October 28, 2025, <https://medium.com/>
6. RESTful API Design Best Practices – Mozilla Developer Network (MDN), accessed on October 28, 2025, https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Introduction
7. Web Application Architecture: Principles and Best Practices – FreeCodeCamp, accessed on October 28, 2025, <https://www.freecodecamp.org/news/>
8. Responsive Web Design with React – CSS-Tricks, accessed on October 28, 2025, <https://css-tricks.com/>
9. Google Maps API Documentation – Geolocation and Nearby Store Integration, accessed on October 28, 2025, <https://developers.google.com/maps/documentation>
10. Designing Scalable Flask Applications with MongoDB – DigitalOcean Tutorials, accessed on October 28, 2025, <https://www.digitalocean.com/community/tutorials>
11. Medicine Availability and Accessibility in Urban India – National Institute of Health (NIH), accessed on October 28, 2025, <https://pubmed.ncbi.nlm.nih.gov/>
12. How to Build a Pharmacy Locator Web Application – GeeksforGeeks, accessed on October 28, 2025, <https://www.geeksforgeeks.org/build-a-pharmacy-locator-web-app/>
13. Web Development Frameworks Comparison – Flask, Django, Express, accessed on October 28, 2025, <https://www.analyticsvidhya.com/>
14. Frontend and Backend Integration Using REST APIs – Tutorialspoint, accessed on October 28, 2025, <https://www.tutorialspoint.com/restful/index.htm>
15. Best UI Design Principles for Web-Based Dashboards – UX Design Institute, accessed on October 28, 2025, <https://www.uxdesigninstitute.com/>
16. Pharmacy Management System – IEEE Research Papers, accessed on October 28, 2025, <https://ieeexplore.ieee.org/>
17. Improving Healthcare Accessibility through Web Technology – ResearchGate, accessed on October 28, 2025, <https://www.researchgate.net/>
18. Authentication and Access Control in Flask Applications – Real Python, accessed on October 28, 2025, <https://realpython.com/>
19. MongoDB Atlas – Cloud Database Service for Flask Applications, accessed on October 28, 2025, <https://www.mongodb.com/atlas>
20. Medicine Data Management Systems – ScienceDirect Journal, accessed on October 28, 2025, <https://www.sciencedirect.com/>
21. Building Scalable Web Applications with Flask – Towards Data Science, accessed on October 28, 2025, <https://towardsdatascience.com/>
22. Healthcare Information Systems and Web Apps – SpringerLink, accessed on October 28, 2025, <https://link.springer.com/>
23. Creating Interactive Web Apps using React – Smashing Magazine, accessed on October 28, 2025, <https://www.smashingmagazine.com/>
24. Pharmacy Locator App Development Guide – Appinventiv Blog, accessed on October 28, 2025, <https://appinventiv.com/blog/>

25. MongoDB CRUD Operations – W3Schools, accessed on October 28, 2025, <https://www.w3schools.com/mongodb/>
26. User Interface Design Guidelines for Healthcare Systems – Elsevier, accessed on October 28, 2025, <https://www.elsevier.com/>
27. Web Application Security Best Practices – OWASP Foundation, accessed on October 28, 2025, <https://owasp.org/>
28. CSS Flexbox and Grid Layout – MDN Documentation, accessed on October 28, 2025, https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout
29. Pharmaceutical Information Retrieval System – Academia.edu, accessed on October 28, 2025, <https://www.academia.edu/>
30. India's Digital Health Mission: Connecting Healthcare Online – National Health Authority, accessed on October 28, 2025, <https://abdm.gov.in/>

ANNEXURE I - SAMPLE CODE

Sample 1: Database Connection using Mongoose

```
// Connect to MongoDB using Mongoose

const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/', {
  dbName: 'meds',
  useNewUrlParser: true,
  useUnifiedTopology: true
}, err => err

  ? console.log(err)

  : console.log('Connected to database successfully')
);
```

Sample 2: Defining Schemas for Data Storage

```
// Schema definitions for Admin, Medical Store, and Medicine
```

```
const AdminSchema = new mongoose.Schema({  
  name: String,  
  address: String,  
  contact: String,  
  email: { type: String, unique: true }  
});
```

```
const MedicalSchema = new mongoose.Schema({  
  storename: String,  
  owner: String,  
  address: String,  
  contact: String,  
  license: String,  
  licensePath: String, // File upload path  
  email: { type: String, unique: true }  
});
```

```
const MedicineSchema = new mongoose.Schema({  
  medname: String,  
  company: String,  
  license: String,  
  des: String,
```

```
    u_price: String,  
    type: String,  
    medical_email: String  
  });
```

```
// Creating Models
```

```
const AdminData = mongoose.model('admindata', AdminSchema);  
const MedicalData = mongoose.model('medicaldata', MedicalSchema);  
const MedicineData = mongoose.model('medicinedata', MedicineSchema);
```

Sample 3: Registering a New Medical Store with License Upload

```
// Using multer to upload medical store license files
```

```
const multer = require('multer');  
const fs = require('fs');
```

```
// Create directory if not exists
```

```
const uploadDir = 'uploads/licenses';  
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });
```

```
// Configure multer
```

```
const storage = multer.diskStorage({  
  destination: (req, file, cb) => cb(null, uploadDir),  
  filename: (req, file, cb) => cb(null, Date.now() + '-' + file.originalname)  
});
```

```
const upload = multer({ storage: storage });
```

Sample 3: Registering a New Medical Store with License Upload

```
// Route for Medical Store Registration
```

```
app.post("/register_medical", upload.single('licenseFile'), async (req, res) => {  
  
  try {  
  
    const { sname, owner, address, contact, lno, email, password, cpassword } =  
    req.body;  
  
    if (!req.file) return res.status(400).json({ message: "License file required" });  
  
    const medical = new MedicalData({  
  
      storename: sname,  
  
      owner,  
  
      address,  
  
      contact,  
  
      license: lno,  
  
      licensePath: req.file.path,  
  
      email  
  
    });  
  
    await medical.save();  
  
    res.json({ success: true, message: "Medical store registered successfully!" });  
  
  } catch (e) {
```

```
        res.status(500).json({ success: false, message: e.message });
    }
});
```

Sample 4: Registering and Displaying Medicines

// Register a new medicine under logged-in medical store

```
app.post("/register_medicine", async (req, res) => {
    try {
        if (!session.isLoggedIn)
            return res.json({ msg: "Login required" });

        const { name, com, lic, desp, uprice, type } = req.body;
        const email = session.email;

        const newMed = new MedicineData({
            medname: name,
            company: com,
            license: lic,
            des: desp,
            u_price: uprice,
            type,
            medical_email: email
        });
    }
});
```

```

    await newMed.save();

    res.json({ msg: "Medicine added successfully" });

  } catch (e) {

    res.json({ msg: "Error saving medicine" });

  }

});

// Display medicines added by a particular store

app.get("/show_medicine", async (req, res) => {

  const medicines = await MedicineData.find({ medical_email: session.email });

  res.json(medicines);

});

```

Sample 5: Medicine Finder – Search with Store Info

```

// Search medicine and get corresponding store details

app.post("/get_med", async (req, res) => {

  try {

    const searchTerm = req.body.mediname;

    const medicines = await MedicineData.aggregate([

      {

        $match: {

```

```
$or: [
  { medname: { $regex: searchTerm, $options: 'i' } },
  { des: { $regex: searchTerm, $options: 'i' } },
  { company: { $regex: searchTerm, $options: 'i' } }
]
}
},
{
  $lookup: {
    from: "medicaldatas",
    localField: "medical_email",
    foreignField: "email",
    as: "storeInfo"
  }
},
{ $unwind: "$storeInfo" },
{
  $project: {
    medname: 1,
    company: 1,
    des: 1,
    u_price: 1,
    type: 1,
```



```
        storename: "$storeInfo.storename",  
        contact: "$storeInfo.contact",  
        address: "$storeInfo.address"  
    }  
}  
]);
```

```
    res.json(medicines);  
} catch (e) {  
    res.status(500).json({ msg: "Error fetching medicine details" });  
}  
});
```